BACKEND ENGINEERING REFERENCE

Translate App API

A NestJS + MySQL backend for a travel-companion app: expense tracking with receipt OCR, AI trip planning, a live voice interpreter, place discovery, and app-wide response translation — wrapped in Firebase auth and a Konnect/Moyasar subscription system.

Runtime Node.js / NestJS 11 **Database** MySQL (TypeORM) **Auth** Firebase Authentication

Docs generated 2026-08-26

What this service does

The repository is named `translate-app` and the Swagger title reads "Translate App API," but the actual product is a travel companion: users track spending on trips, scan paper receipts into structured data, plan itineraries with AI, talk to locals through a live speech interpreter, discover nearby places, and pay for a subscription. Every JSON response is transparently translated into the caller's saved language — that's the thread tying the modules together.

- **Expense tracking** — categorized expenses, optionally linked to a trip, with merchant geolocation captured from receipts.
- **Receipt OCR** — photographed receipts are parsed by an external Gemini-based microservice into merchant, line items, totals, and tax.
- **Trip planning** — an external AI service drafts an itinerary and budget, which the backend enriches with real Google Places data.
- **Voice interpreter** — two people speaking different languages get live speech-to-text → translate → text-to-speech turns.
- **Explore** — Google Places-backed nearby search, proactive "you're near something new" check-in notifications, and AI place narration.
- **Subscriptions** — a 3-day free trial converts to a paid plan via Konnect (Tunisia, TND) or Moyasar (Gulf currencies).
- **Support chat** — SQL-backed conversations mirrored into Firebase Realtime Database for live delivery.

CROSS-CUTTING

Architecture

Bootstrapped in `src/main.ts`; global wiring lives in `src/app.module.ts`. 24 feature modules hang off one root module — nearly all of it composes the same handful of global mechanisms below.

Bootstrap (`main.ts`)

- Trusts only the loopback proxy (`trust proxy: 'loopback'`) so rate-limiting sees the real client IP behind nginx.
- Swagger UI at `/api-docs` with persisted Bearer auth.
- CORS open; port from `process.env.PORT`, default `3000`.

Global providers (`app.module.ts`)

MECHANISM	PROVIDER	EFFECT
APP_GUARD	`ThrottlerGuard`	Rate limiting on every route — default 120 requests / 60s, tightened per-route where noted.
APP_INTERCEPTOR	`ResponseFormatInterceptor`	Wraps every response as `{status_code, title, body, data}` and translates `title`/`body` into the caller's language. Skips 204 responses.
APP_FILTER	`HttpExceptionFilter`	Same response envelope for errors, same translation, respects an `AppException.translate` opt-out flag.

Other globals

- `ConfigModule` — global, loads `.env`.
- `ServeStaticModule` — serves uploaded files under `/files` from the `files/` directory (profile images, receipt sources, etc).
- `CacheModule` — global, in-memory, TTL 300s / max 100 entries. Backs the password-reset OTP flow.
- `TypeOrmModule` — MySQL via `mysql2`; `synchronize` is only `true` when `NODE_ENV !== 'production'`; `autoLoadEntities` is on.
- `EventEmitterModule` — decouples payment success from notification delivery.

The translation thread

A `@Translate({ ignoreKeys })` decorator + `TranslateInterceptor` is layered onto almost every list/get endpoint across the app. It recursively walks the response JSON and translates string values (skipping any key named in `ignoreKeys` — typically ids, dates, colors, numeric fields) into the language on the caller's user record, via an external translator

microservice. Translation is treated as a non-critical enhancement everywhere: any failure is caught and the original text is returned rather than the request failing.

SRC/AUTH

Auth

Firebase Authentication is the identity provider. There is no local JWT issuance — every protected route verifies a Firebase ID token server-side. A mirrored `User` row lives in MySQL, keyed by `firebaseUid`, and gets created or synced the first time a token is seen.

Guards

GUARD	BEHAVIOR
`FirebaseAuthGuard`	Requires `Authorization: Bearer <idToken>`, verifies via `getAuth().verifyIdToken`, then finds-or-creates the matching SQL user and attaches it to the request.
`OptionalFirebaseAuthGuard`	Same check, but swallows failures instead of blocking — used on public listings (currencies, countries, FAQs) that still personalize/translate for a logged-in caller.
`AdminGuard`	Requires `user.isAdmin()` (role === 'admin'). Always stacked after FirebaseAuthGuard.
`SubscriptionGuard`	Checks `user.hasActiveAccess()` (active trial or paid subscription). **Defined but not attached to any controller** — see Known Issues.

Password reset — custom OTP flow

Firebase's built-in reset flow is bypassed in favor of a 3-step, cache-backed OTP exchange, each step independently rate-limited:

STEP	ROUTE	RATE LIMIT	BEHAVIOR
1	`POST /auth/send-reset-mail`	3 / 15 min	Generates a 4-digit code, caches it 5 min, emails an Arabic RTL template. Always returns a generic 200 to avoid confirming whether an email exists.
2	`POST /auth/validate-reset-mail`	5 / 5 min	Checks the OTP, issues a single-use `resetPassToken` cached 10 min.
3	`POST /auth/reset-password`	5 / 10 min	Validates the token, calls `getAuth().updateUser(firebaseUid {password})`.

Routes

METHOD	PATH	GUARD	PURPOSE
POST	/auth/register	public	Email/password sign-up via the Firebase Identity Toolkit REST API; syncs the SQL user.
POST	/auth/login	public	Email/password sign-in.
POST	/auth/oauth	public	Google / Facebook IDP login (`signInWithIdp`).
POST	/auth/refresh	public	Exchanges a refresh token via `securetoken.googleapis.com`.
POST	/auth/firebase-token	FirebaseAuthGuard	Mints a custom Firebase token embedding the `role` claim, consumed by Realtime Database security rules.

All register/login/oauth responses return `{ user, idToken, refreshToken }`.

SRC/USER

Users & Admin

User entity

FIELD	TYPE	NOTE
id	uuid	Prim
role	enum	user
firebaseUid	string	Uniq
email	string	Uniq
displayName / photoURL	string	phot plac
provider	enum	pass facel
isActive	boolean	Soft from
currentCountryId / currencyId / languageId	FK → Country / Currency / Language	All e
subscriptionStatus	enum	trial canc
trialEndsAt / subscriptionEndsAt	datetime	Trial inser
paymentCustomerId	string	Prov refer

Instance methods: `hasActiveAccess()` (trial or active-and-unexpired) and `isAdmin()`.

Routes

METHOD	PATH	GUARD	PURPOSE
GET	/users/me	FirebaseAuthGuard	Current user profile.
PATCH	/users/me	FirebaseAuthGuard	Multipart update; prof saved to `./files/pr` `images` via multer.

METHOD	PATH	GUARD	PURPOSE
DELETE	/users/me	FirebaseAuthGuard	Requires a matching confirmEmail in the
GET	/admin/users	Firebase + Admin	Paginated list.
GET	/admin/users/:id	Firebase + Admin	Single user.
PATCH	/admin/users/:id	Firebase + Admin	Edit profile fields.
PATCH	/admin/users/:id/role	Firebase + Admin	Promote/demote adm
PATCH	/admin/users/:id/active	Firebase + Admin	Enable/disable accou
DELETE	/admin/users/:id	Firebase + Admin	SQL-only delete — doe touch the Firebase ac
GET	/admin/stats/overview	Firebase + Admin	Combined KPI snapsh
GET	/admin/stats/users-per-country	Firebase + Admin	
GET	/admin/stats/users-per-currency	Firebase + Admin	
GET	/admin/stats/users-per-provider	Firebase + Admin	
GET	/admin/stats/signups-over-time?days=	Firebase + Admin	
GET	/admin/stats/revenue	Firebase + Admin	
GET	/admin/stats/dashboard	Firebase + Admin	All of the above, comb

ADMIN BOOTSTRAP

`AdminBootstrapService` runs on module init: if `ADMIN_EMAIL` / `ADMIN_PASSWORD` are set and no matching SQL admin exists, it finds-or-creates the Firebase user, mirrors an admin row in MySQL, and sets the Firebase custom claim `{ role: 'admin' }`.

SRC/PAYMENT

Payments & Subscriptions

A strategy pattern (`IPaymentProviderStrategy`: `initPayment` / `verifyPayment`) picks a provider by currency code.

Providers

PROVIDER	CURRENCIES	NOTES
Konnect	`TND only`	Tunisia. Converts to millimes (×1000). Calls sandbox POST `/payments/init-payment` with wallet/webhook/success/fail URLs.
Moyasar	`SAR, USD, AED, EUR, KWD, BHD, OMR`	Gulf. Per-currency minor-unit multiplier (100, or 1000 for 3-decimal currencies). Basic-auth with the secret key; creates an "invoice" via POST `/invoices`.

Entities

ENTITY	KEY FIELDS
PaymentPlan	name, description, durationMonths, basePrice + baseCurrencyId, isActive.
Payment	userId, paymentPlanId, amount + currencyId (snapshotted at charge time), status (pending/succeeded/failed/cancelled/refunded), provider (konnect/moyasser/manual), providerTransactionId.

Webhook flow

`handleWebhook(provider, ref)` is idempotent — it returns early if the payment already resolved. Otherwise it verifies with the provider strategy and, inside a pessimistic-write DB transaction, updates the payment status and — on success — extends `user.subscriptionEndsAt` (stacking onto any still-active period) and flips `subscriptionStatus` to active, then emits a notification event.

Routes

METHOD	PATH	GUARD	PURPOSE
POST	/payment/checkout/:planId	FirebaseAuthGuard	Converts the plan's b to the user's currenc a pending payment, r checkout URL.

METHOD	PATH	GUARD	PURPOSE
POST	/payment/:paymentId/verify	FirebaseAuthGuard	Manual re-check aga provider.
GET	/payment/webhook/konnect?payment_ref=	public	Provider callback.
POST	/payment/webhook/moyasar?paymentId=	public	Provider callback.
GET	/payment-plans	FirebaseAuthGuard	Active plans, priced i caller's own currency
GET	/payment-plans/raw	none	All plans, raw base-c price. **No guard appl**
GET	/payment-plans/raw/:id	none	Single plan, raw price **guard applied.**
POST	/payment-plans	none	Create a plan. **No gua applied.**
PATCH	/payment-plans/:id	none	Update a plan. **No gu applied.**
DELETE	/payment-plans/:id	none	Delete a plan. **No gua** applied

FLAG

Every other admin-only surface in this API is protected by `FirebaseAuthGuard` + `AdminGuard`. The plan-management routes above are the one exception — see Known Issues.

SRC/TRIP

Trips

One active trip per user at a time. Creating or activating a trip auto-archives whichever other trip was previously active.

Trip entity

FIELD	TYPE
name, destination, notes	string
country / countryId	string + F
travelers	int
startDate / endDate	date
totalAmount / currencyId	decimal +
status	enum
interests, categoryLimits, reminders, monuments, itinerary, budgetGuidance	json
aiSummary	text
userId	FK → User

AI plan generation

generatePlan sends the trip context, spending breakdown, and the user's expense categories to an external AI endpoint ({OCR_SERVICE_URL}/ai/trip-plan). The returned monuments/itinerary items are then enriched with real coordinates and Maps links via a Google Places text search, and any AI-suggested category budgets are scaled down proportionally if they exceed the trip's total budget before being persisted.

Routes

METHOD	PATH	GUARD	PURPOSE
GET	/trips	FirebaseAuthGuard	List the caller's trips.
GET	/trips/:id/expenses	FirebaseAuthGuard	Expenses tagged to the trip, or falling in its date range.

METHOD	PATH	GUARD	PURPOSE
POST	/trips	FirebaseAuthGuard	Create; archives any other active trip.
PATCH	/trips/:id	FirebaseAuthGuard	Edit.
DELETE	/trips/:id	FirebaseAuthGuard	Soft-delete: sets status to archived.
POST	/trips/:id/generate-plan	FirebaseAuthGuard	Runs the AI planning pipeline above.
POST	/trips/:id/activate	FirebaseAuthGuard	Marks active, demotes the previous active trip.

Expenses & Categories

Expense entity

FIELD	TYPE	NOTES
amount	decimal (string)	
description, shop	string	
date	timestamp	Indexed with use
currencyId	FK → Currency (eager)	
categoryId	FK → ExpenseCategory	Cascade delete; i userId.
tripId	FK → Trip, nullable	SET NULL on trip
userId	FK → User	Cascade delete.
googlePlaceId, merchantAddress, googleMapsUri	string	Populated when originates from a matched mercha
merchantLatitude / merchantLongitude	decimal(10,7)	
merchantMatchConfidence	smallint (0–100)	

ExpenseCategory entity

name (unique per user, composite index with userId), icon, color (hex), description, userId → User (cascade). Has many Expense.

Routes

METHOD	PATH	GUARD	PURPOSE
GET	/expenses/statistics	FirebaseAuthGuard	Declared before :id so it isn't shadowed by the id route.
POST	/expenses	FirebaseAuthGuard	Create manually.
POST	/expenses/from-receipt	FirebaseAuthGuard	Create from a previously saved Receipt.
GET	/expenses	FirebaseAuthGuard	List, translated.

METHOD	PATH	GUARD	PURPOSE
GET	/expenses/:id	FirebaseAuthGuard	
PATCH	/expenses/:id	FirebaseAuthGuard	
DELETE	/expenses/:id	FirebaseAuthGuard	Returns 204.

/expense-categories — standard CRUD (GET, GET :id, POST, PATCH :id, DELETE :id), all FirebaseAuthGuard, scoped to the caller

SRC/RECEIPTS

Receipts (OCR)

A photographed receipt is sent to an external "Gemini Receipt OCR" microservice and parsed into a structured `Receipt` with line items.

Entities

Receipt — userId (nullable, cascade), documentType, language, country, merchant, address, phone, invoiceNumber, date, time, currency (string code, not FK), subtotal/tax/total (float), paymentMethod, rawText, originalFileName, imageMimeType. Has many **ReceiptItem** (name, description, quantity, unitPrice, totalPrice — eager, cascade).

Routes

METHOD	PATH	GUARD	PURPOSE
POST	`/receipts/merchant-match`	FirebaseAuthGuard (10/min)	Matches a n real nearby
POST	`/receipts/extract`	FirebaseAuthGuard	Multipart im (jpg/jpeg/pr 10MB) → O(save.
POST	`/receipts/receipt-recommendations/:receiptId`	FirebaseAuthGuard	AI-suggeste type/descri a receipt to
GET	`/receipts`	FirebaseAuthGuard	List, transla (ids/dates/r excluded).
GET	`/receipts/:id`	FirebaseAuthGuard	
PATCH	`/receipts/:id`	FirebaseAuthGuard	
DELETE	`/receipts/:id`	FirebaseAuthGuard	

A separate `guest-receipts.controller.ts` exposes an unauthenticated variant of parts of this flow.

SRC/INTERPRETER

Interpreter (Voice)

Live voice-to-voice interpreting between a traveler and a local speaker of a different language, structured as a conversation of alternating turns.

InterpreterConversation — userId, travelerLanguage, localLanguage, title. Has many **InterpreterTurn** — speaker (traveler | local), sourceLanguage, targetLanguage, sourceText, translatedText, sourceAudioUrl/translatedAudioUrl + mime types.

Routes

METHOD	PATH	GUARD	PURPOSE
POST	`/interpreter/conversations`	FirebaseAuthGuard	Start a conversation.
GET	`/interpreter/conversations`	FirebaseAuthGuard	List.
GET	`/interpreter/conversations/:id`	FirebaseAuthGuard	
POST	`/interpreter/conversations/:id/turns`	FirebaseAuthGuard	Multipart audio (must `audio/*`, max 10MB) speech-to-text → tran text-to-speech.
DELETE	`/interpreter/conversations/:id`	FirebaseAuthGuard	

Backed by an external microservice at `TRANSLATOR_SERVICE_URL` (`/ai/speech-to-text`, `/ai/text-to-speech`, `/ai/translate`). A parallel `guest-interpreter.controller.ts` offers an unauthenticated variant, mirroring the pattern used elsewhere in the app.

SRC/EXPLORE

Explore (Places)

Google Places-backed discovery, plus location-triggered "you're near something new" notifications.

METHOD	PATH	GUARD	PURPOSE
POST	/explore/nearby	FirebaseAuthGuard (20/min)	Nearby attractions/muse ranked by distan
POST	/guest/explore/nearby	public (10/min)	Same, unauthent
POST	/explore/check-in	FirebaseAuthGuard	Geolocation chec undiscovered pla and creates a pu cooled down 1hr per place.
GET	/guest/explore/place/:placeId	public	Place details.
POST	/guest/explore/place/:placeId/narration	public	Translated, spok a place.
POST	/guest/explore/place/:placeId/description	public	AI-generated pla

Requires `GOOGLE_MAPS_API_KEY`. Photo resolution goes through a shared helper (`src/common/google-place-photo.ts`) also used by Trip's place enrichment.

SRC/CHAT

Chat & Support

SQL is the source of truth; every write is mirrored into Firebase Realtime Database so clients subscribe directly with the Firebase client SDK — **no persistent socket connection to this server is required**. A `ChatGateway` Socket.IO namespace (`/support-ws`) still exists in the codebase but is explicitly superseded by this design per the module's own README.

Entities

Conversation — one per user (unique userId index), userDisplayName, lastMessagePreview, lastMessageAt, unreadByAdmin/unreadByUser counters. **Message** — conversationId (cascade), senderId (Firebase UID), senderRole (user/admin), body, isRead, composite index [conversationId, createdAt].

Routes

METHOD	PATH	GUARD	PURPOSE
GET	`/chat/conversations/me`	FirebaseAuthGuard	Get-or-crea thread.
GET	`/chat/conversations/:id/messages?before&limit`	FirebaseAuthGuard	Paginated h
POST	`/chat/conversations/:id/messages`	FirebaseAuthGuard	Send; mirro the SQL wri
PATCH	`/chat/conversations/:id/read`	FirebaseAuthGuard	Mark read.
GET	`/chat/conversations`	Firebase + Admin	List every c view).

Ownership is enforced on every non-admin route — a caller can only touch their own conversation. RTDB data shape: `chats/conversations/{id}`, `chats/messages/{conversationId}/{messageId}`, `chats/userIndex/{userId}`, `chats/typing/{conversationId}/{role}`. All RTDB writes are locked to `false` in `firebase-database.rules.json` except `typing` — every real write happens server-side through the Admin SDK, which bypasses rules; clients only ever read.

CAVEAT (FROM THE MODULE README)

Admin custom claims only apply to a Firebase ID token issued *after* the claim was set — a user promoted to admin mid-session needs a forced token refresh before RTDB rules recognize the new role.

SRC/NOTIFICATION · SRC/MAILER

Notifications & Mail

Notification — userId nullable (null = broadcast to all devices), title, body, data (json), source (manual/event), read, pushSent. **DeviceToken** — userId, token (unique), platform (ios/android/web).

On create, a notification is persisted then pushed via FCM (`messaging.sendEachForMulticast`) unless explicitly suppressed; tokens FCM reports as unregistered are auto-pruned. Push failures are logged, never surfaced to the caller. A `@OnEvent` listener creates a notification when Payments emits a subscription-activation event. `checkCurrentCountry` compares a device-detected country against the user's declared country and, if mismatched, raises a suggestion notification (deduplicated).

METHOD	PATH	GUARD	PURPOSE
POST	/notifications	FirebaseAuthGuard	Create for self.
POST	/notifications/check-country	FirebaseAuthGuard	
GET	/notifications/me	FirebaseAuthGuard	
GET	/notifications/:id	FirebaseAuthGuard	
PATCH	/notifications/:id	FirebaseAuthGuard	
PATCH	/notifications/:id/read	FirebaseAuthGuard	
PATCH	/notifications/me/read-all	FirebaseAuthGuard	
DELETE	/notifications/:id	FirebaseAuthGuard	
POST	/notifications/devices	FirebaseAuthGuard	Register/refresh an FCM token.
DELETE	/notifications/devices/:token	FirebaseAuthGuard	

Mailer wraps `@nestjs-modules/mailer` over SMTP; its only current consumer is the Arabic password-reset OTP email in Auth.

SRC/TRANSLATION

Translation Layer

The same external microservice used by Interpreter (`TRANSLATOR_SERVICE_URL`, endpoints `/ai/translate` and `/ai/translate-json`) backs both direct translation endpoints and the app-wide `@Translate` interceptor described under Architecture. Target language resolves as: explicit `target` param → the caller's saved language → English. Failures fail open — original text is returned.

METHOD	PATH	GUARD	PURPOSE
POST	`/translation/text`	FirebaseAuthGuard	
POST	`/translation/json`	OptionalFirebaseAuthGuard	Overridden to allow guests.
POST	`/guest/translation/json`	`public`	No user context — falls back to English unless `target` is passed explicitly.

Reference Data

Currency

Currency — code (3-char, unique), name, isActive.

METHOD	PATH	GUARD	PURPOSE
GET	/currencies	Optional auth	Translate
GET	/currencies/convert?amount&fromCurrencyId&toCurrencyId	public	
GET	/currencies/:id	Optional auth	
POST / PATCH :id / DELETE :id	/currencies	Firebase + Admin	

Country

Country — name, alpha2Code (unique), alpha3Code (unique), flag emoji, region, capital, currencyId/languageId → Currency/Language (both eager), isActive.

METHOD	PATH	GUARD	PURPOSE
GET	/countries	Optional auth	Translated list.
GET	/countries/:id/preferences	Optional auth	Default currency/language for a country.
GET	/countries/:id	Optional auth	
POST / PATCH / DELETE	/countries	Firebase + Admin	

Language

Minimal entity: id, code (unique), name.

METHOD	PATH	GUARD	PURPOSE
GET	/languages	public	
GET	/languages/search?q=	public	
GET	/languages/:id	public	
POST / PATCH / DELETE	/languages	Firebase + Admin	

FAQ

FaqCategory (order, isActive) has many **Faq**. Deleting a category uncategorizes its FAQs rather than deleting them.

METHOD	PATH	GUARD	PURPOSE
GET	/faqs, /faqs/categories, ...	Optional auth	Public, translated reads.
POST / PATCH / DELETE	/faqs/...	Firebase + Admin	409 on duplicate category name.

MYSQL · TYPEORM

Database & Migrations

MySQL via `mysql2`. `synchronize` is enabled outside production, so local/staging schemas auto-sync from entities; production schema changes are applied by hand-written SQL migration files in `migrations/` rather than TypeORM's own migration runner:

FILE	ADDS
`20260809_interpreter.sql`	Interpreter conversations & turns tables.
`20260811_expense_merchant_place.sql`	Merchant/place columns on expenses (address, lat/lng, Google place id, match confidence).
`20260817_receipt_ownership.sql`	Receipt → user ownership FK.
`20260823_trips.sql`	Trips table.
`20260824_trip_country.sql`	Trip → country FK.
`20260826_country_language.sql`	Country → language FK.

`firebase-database.rules.json` (repo root) governs the Realtime Database mirror used by Chat — see Chat & Support.

CONFIGURATION

Environment Variables

Names only — see Known Issues regarding the live `.env` file in this repo.

App

- PORT
- NODE_ENV
- APP_BASE_URL

Database

- DB_TYPE
- DB_HOST
- DB_PORT
- DB_USERNAME
- DB_PASSWORD
- DB_NAME

Firebase

- FIREBASE_PROJECT_ID
- FIREBASE_CLIENT_EMAIL
- FIREBASE_PRIVATE_KEY
- FIREBASE_DATABASE_URL
- FIREBASE_WEB_API_KEY

Admin bootstrap

- ADMIN_EMAIL
- ADMIN_PASSWORD

Payments

- KONNECT_BASE_URL
- KONNECT_API_KEY
- KONNECT_RECEIVER_WALLET_ID
- MOYASAR_BASE_URL
- MOYASAR_SECRET_KEY

Mail (SMTP)

- MAIL_HOST
- MAIL_PORT
- MAIL_SECURE
- MAIL_USER
- MAIL_PASS
- MAIL_FROM_NAME

Maps / Places

- GOOGLE_MAPS_API_KEY

AI microservices

- TRANSLATOR_SERVICE_URL (default localhost:8000)
- OCR_SERVICE_URL (default localhost:8005)
- OCR_SERVICE_API_KEY

OCR_RECEIPT_ENDPOINT (default /ai/receipt)

FOR THE TEAM

Known Issues

1 · PAYMENT PLAN ADMIN ROUTES ARE UNGUARDED

`PaymentPlanController`'s `GET /payment-plans/raw`, `GET /payment-plans/raw/:id`, `POST /payment-plans`, `PATCH /payment-plans/:id`, and `DELETE /payment-plans/:id` carry no guard at all — anyone can list, create, edit, or delete subscription plans. Every comparable admin surface elsewhere uses `FirebaseAuthGuard` + `AdminGuard`; this one should match.

2 · PAYMENTPLANCONTROLLER IMPORTS ITS SERVICE FROM A SPEC FILE

`src/payment/payment-plan.controller.ts` imports `PaymentPlanService` from `'./payment-plan.service.spec'` instead of `'./payment-plan.service'`. It works only because the spec file happens to export a class of the same name — but it means production wiring depends on a test file.

3 · SUBSCRIPTIONGUARD IS DEFINED BUT NEVER APPLIED

`src/auth/subscription.guard.ts` exists and checks `hasActiveAccess()`, but no controller in the codebase attaches it. If trial/paid gating is intended on any route, it currently isn't enforced there.

4 · DUPLICATE CURRENTUSER DECORATOR

Two identical implementations exist — `src/auth/current-user.decorator.ts` and `src/common/decorators/current-user.decorator.ts` — both reading `request.user`, used interchangeably across controllers. Harmless, but worth consolidating.

5 · STRAY BACKUP FILE

`src/user/user.module copy.ts` (note the space) looks like an accidental duplicate left in the source tree rather than a real module.

6 · LIVE-LOOKING SECRETS IN THE WORKING DIRECTORY'S .ENV

The repo's `.env` (not a checked-in `.env.example` — none exists) holds what appear to be real credentials: a full Firebase service-account private key, a Konnect API key, a Moyasar `sk_test_...` secret, an SMTP app password, and a Google Maps key. `.gitignore` does correctly exclude `.env`, so nothing has been committed — but consider adding a checked-in `.env.example` with variable names only, and rotating any of these that may have been shared outside version control.

Generated from the repository at `C:\Users\akihi\Documents\translate-app` on 2026-08-26 by reading the source directly (modules, controllers, entities, migrations). Not derived from the boilerplate README, which is unmodified Nest starter text.